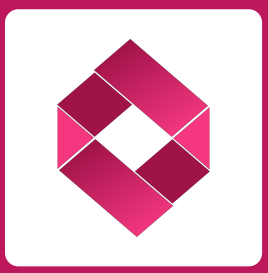
COMPLETE PROJECT DOCUMENTATION



H.E.X

Health • Education • eXploration
A JavaFX desktop companion for essential city services

Find hospitals, tests, blood, books, tutors and parking - in one app

PROJECT TYPE	Java desktop application (JavaFX + MySQL)
PLATFORM	Java 21, JavaFX 21, Maven, JDBC
USER ROLES	Guest | Registered user | Department admin
MODULES	Health | Education | Exploration (Parking) | Chat
LEAD DEVELOPER	Md Ahbab Hamid Khan (+ team)
DOCUMENT	Version 1.0 - 3 November 2026

Prepared from the complete project source code, database scripts and files.

Table of Contents

1. Executive Summary . . . 4

Why it matters . . . 4

2. Problem Statement and Objectives . . . 4

2.1 Problem . . . 4

2.2 Objectives . . . 4

3. Scope and Implementation Status . . . 5

4. Users and Roles . . . 6

5. Features in Detail . . . 6

5.1 Home and navigation . . . 6

5.2 Main dashboard (GuestScreen) . . . 6

5.3 Health module . . . 6

5.4 Education module . . . 7

5.5 Exploration module - Parking . . . 7

5.6 Communication - chat . . . 7

5.7 Administration consoles . . . 7

5.8 Account and help . . . 8

6. System Architecture . . . 8

6.1 Technology stack . . . 8

6.2 Source packages . . . 9

6.3 Screen navigation map . . . 9

7. Chat and Socket Server . . . 9

8. Database Design . . . 10

8.1 Data dictionary . . . 11

8.2 Seed data supplied . . . 11

9. Key Workflows . . . 11

9.1 Find ICU beds . . . 11

9.2 Administrator updates bed count . . . 12

9.3 Verify a tutor . . . 12

9.4 Chat with an administrator . . . 12

10. Installation and Run Guide . . . 12

Prerequisites . . . 12

Steps . . . 12

11. Security and Quality Review . . . 13

11.1 Strengths . . . 13

11.2 Findings and recommendations . . . 13

12. Test Cases . . . 14

13. Roadmap . . . 14

14. What We Built - Team and Deliverables . . . 15

15. Presentation Guide and Viva Questions 15

15.1 Suggested 7-minute demo 15

15.2 Likely questions and model answers 15

Appendix - Glossary 16

1. Executive Summary

H.E.X (*Health, Education, eXploration*, window title "Health Education and Co.") is a desktop application that puts everyday essential services behind one friendly interface. A person can search for **hospital beds and ICU availability**, compare **medical test prices**, locate **blood banks and pharmacies**, browse a **library catalogue**, find **verified tutors** and look up **parking slots**. Signed-in users can chat directly with the administrators who maintain the data, and each administrator manages only their own department.

The application is written in **Java 21 with JavaFX** (FXML screens styled with CSS), uses **MySQL over JDBC** for storage and contains a small **multi-threaded TCP socket server** for the chat feature. It is built with **Maven** and packaged as a Java module.

3	4	20	21	9
Service domains	User roles	Java classes	FXML screens	Database tables

Why it matters

- **Speed in urgent situations** - bed, ICU and blood availability can be checked in seconds.
- **Transparency** - test prices from several hospitals are listed side by side.
- **Trust** - tutors appear to students only after an administrator verifies them.
- **Clear ownership** - Health, Education and Others (parking) administrators each control only their data.
- **No sign-in barrier** - the main services are open to guests; an account is needed only for chat.

2. Problem Statement and Objectives

2.1 Problem

Information that people need daily is scattered: a hospital's free ICU beds are only known by phone, test prices differ widely, finding a verified home tutor depends on word of mouth, and parking spots are found by driving around. There is no single, offline-capable tool that serves these needs together and that allows responsible administrators to keep the data current.

2.2 Objectives

#	Objective	Achieved by
1	Provide quick health-service lookup	ICU & Beds, Health Tests and Blood Bank / Pharmacy search screens (location and test-name search).
2	Support learning resources	Library catalogue and verified-tutor finder.
3	Help people explore the city	Parking slot list with location and area.
4	Keep data accurate	Department-based admin consoles (Health, Education, Others) with add / update / delete actions.
5	Enable communication	User-to-admin chat stored in the database, backed by a socket server.
6	Be easy to use	Modern JavaFX styling (modern.css, animations.css), icon-based dashboard, guest access.

3. Scope and Implementation Status

The table below reports the state of every feature as found in the code, so that demonstrations stay honest and the next development steps are obvious.

Feature	Status	Notes
Home screen with User / Admin / Guest entry	**Working**	HomeScreen.fxml; starts the socket server in a background thread.
Guest dashboard (Health, Education, Exploration)	**Working**	GuestScreen.fxml with icon tiles, account and help menus.
ICU & Beds search by location	**Working**	Joins beds and hospital tables.
Health test search and fee comparison	**Working**	Joins healthtest and hospital.
Blood bank / pharmacy search	**Working**	Shows HasBlood, HasPharmacy, ICUAvailability per hospital.
Library (book keeper)	**Working**	Book, library, availability, price.
Tutor finder (verified only)	**Working**	Student.fxml lists tutors where Adminverified = 1.
Parking slot list	**Working**	Slot, location, area.
User sign-in (dashboard) and user login screen	**Working**	Check username + password in users table.
Department admin login and routing	**Working**	Role ADMIN + department decides which console opens.
Health admin: update ICU status and bed counts	**Working**	HealthFeaturesController.
Education admin: book CRUD, tutor verify / delete	**Working**	EducationScreenController.
Others admin: parking update / delete	**Working**	OthersController.
User ↔ admin chat	**Working (DB-backed)**	Messages saved in messages table; conversation reloads on select / send.
User sign-up	**Needs DB alignment**	UserService inserts an email column that the provided SQL scripts do not create (see Section 11).
Tutor self-registration	Not built	"Sign up / Sign in as a Tutor" button currently reloads the Tutor screen; tutors are added in the database.
Parking reservation, live health data via sockets	Prototype	Server commands exist but return fixed placeholder replies.
Developer "User Manager" console	Partly built	Screen exists; its Sign-in handler is not yet implemented. UserService has the needed CRUD methods.
Health records per user	Prototype	HealthService and health_records (JSON) table exist; no screen yet.

4. Users and Roles

Role	How they enter	Permissions
Guest	"Guest Access" on the home screen	Use Health, Education and Parking services; read-only. Cannot chat.
Registered user	User Login or "Sign In" on the dashboard	Everything a guest can do, plus chat with administrators and account deletion.
Health administrator	Admin Login (role ADMIN, department HEALTH)	View hospitals, beds, tests; update ICU availability and bed numbers; chat with users.
Education administrator	Admin Login (department EDUCATION)	Add / update / delete books; verify or delete tutors; chat with users.
Others administrator	Admin Login (department OTHERS)	View, update, delete parking slots; chat with users.
Developer	Developer Login on the dashboard Admin menu	Intended: view, update and remove user profiles (console under development).

5. Features in Detail

5.1 Home and navigation

HomeScreen shows the H.E.X logo, the tagline *"Health • Education • eXploration"* and three buttons: **User Login**, **Admin Login** and **Guest Access**. When the screen initialises it starts the application's socket server on a background thread, so the chat feature is ready without any extra step. Screens are switched by loading FXML files with `FXMLLoader` and replacing the scene of the current stage (window 1200 x 800).

5.2 Main dashboard (GuestScreen)

The dashboard groups services into three tiles. A menu bar offers **Account** (Sign In, Sign Up, Delete Account, Logout), **Chat** (Open Chat), **Admin** (Admin Login, Developer Login) and **Help** (Contact, Report a Bug, About).

Tile	Buttons	Opens
Health Services	Check Beds, View Tests, Blood Services	ICU & Beds, Health Tests, Blood Bank screens
Education Services	Browse Library, Find Tutors	Library table, Tutor screen
Exploration Services	Find Parking	Parking slot table

5.3 Health module

Screen	User input	Result columns	Source query idea
ICU & Beds	Area / city text	Bed_Available, HospitalID, HospitalName, Location, ICU_Available	beds JOIN hospital WHERE Location LIKE ?
Health Tests	Test name, e.g. Ultrasound, MRI	HospitalName, Location, TestFees	healthtest JOIN hospital WHERE TestName LIKE ?
Blood Bank & Pharmacy	Area / city text	All hospital columns incl. HasBlood, HasPharmacy, ICUAvailability	hospital WHERE Location LIKE ?

All three use **prepared statements with wildcard LIKE search** and a technique worth explaining in a viva: the `TableView` is built *dynamically* from the `ResultSetMetaData` - columns are created from the SQL column names and every row is stored in a `Map<String,String>`. One small method therefore serves any query.

```
ResultSet rs = statement.executeQuery();
for (int i = 0; i < rs.getMetaData().getColumnCount(); i++) {
    TableColumn<Map<String,String>,String> col =
        new TableColumn<>(rs.getMetaData().getColumnName(i + 1));
    col.setCellValueFactory(cd -> new SimpleStringProperty(cd.getValue().get(col.getText())));
    table.getColumns().add(col);
}
while (rs.next()) {
    Map<String,String> row = new HashMap<>();
    for (int i = 1; i <= colCount; i++) row.put(rs.getMetaData().getColumnName(i), rs.getString(i));
    table.getItems().add(row);
}
```

5.4 Education module

- **Library (Book Keeper)** - lists Book, Library, Availability (Yes / No) and Price from `book_keeper`, so a reader sees which library holds a book and what it costs.
- **Find Tutors** - the Tutor screen offers "Looking for tuitions?" and "Looking for coursewise teachers?". *Get Teachers* opens the Student screen, which shows **only tutors with Adminverified = 1** (Name, Subjects, Class, Expected Salary). This protects families from unverified listings.

5.5 Exploration module - Parking

ParkingSlot shows every record of `parking_slot` (Slot number, Location, Area) such as "Shohid Faruk Road - Jatrabari" or "Police Plaza Shopping Complex - Gulshan-2".

5.6 Communication - chat

A signed-in user opens **Chat** and sees the list of administrators (users with role ADMIN). Selecting an administrator loads the conversation; typing a message stores it in the `messages` table with sender, receiver, text, timestamp and read flag. Administrators have a mirrored screen (`AdminCHATTER`) that lists users (role USER). Conversation queries use `(sender,receiver) OR (receiver,sender)` ordered by time, and are run on a background thread with `Platform.runLater` updating the UI - the correct JavaFX threading pattern.

5.7 Administration consoles

Console	Tabs / tables	Actions
Health (Health.fxml)	Hospitals, ICU and beds, Health tests	Update ICU status text by hospital name; update bed count (UPDATE beds JOIN hospital ...); open chat.
Education (Education.fxml)	Tutors, Books	Verify tutor (Adminverified = true), delete tutor, insert / update / delete book; open chat.
Others (Others.fxml)	Parking slots	Update slot / area by location, delete slot; open chat.

Login routing is a compact `switch` on the department returned by `SELECT department FROM users WHERE username=? AND password=? AND role='ADMIN'`:

```
switch (department) {
    case "HEALTH"    -> loadScene("/main/ahbab/main_interface/Admin/Health.fxml");
    case "OTHERS"    -> loadScene("/main/ahbab/main_interface/Admin/Others.fxml");
    case "EDUCATION" -> loadScene("/main/ahbab/main_interface/Admin/Education.fxml");
}
```

5.8 Account and help

- **Sign Up** screen validates empty fields, matching passwords and a minimum of 6 characters, and checks for duplicate usernames.
- **Delete Account** asks for confirmation with a JavaFX confirmation alert.
- **Contact** and **Report a Bug** alerts show the developer's e-mail and can open a mail compose page; **About** opens the project's online document.

6. System Architecture

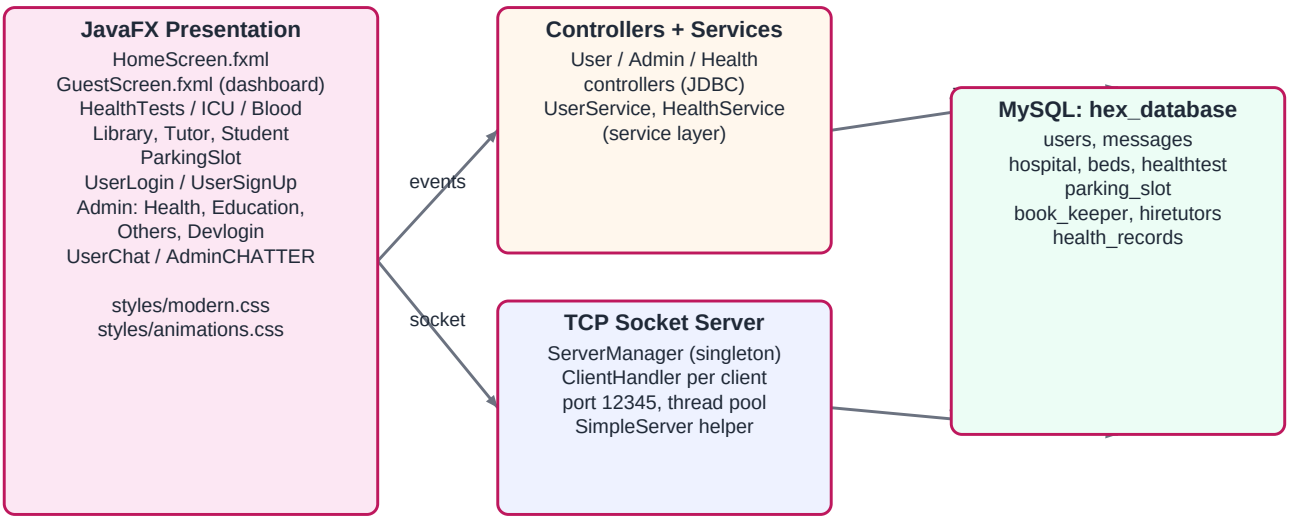


Figure 1 - Layered architecture of H.E.X.

6.1 Technology stack

Item	Technology / version	Purpose
Language	Java 21 (source and target 21)	Application logic
UI toolkit	JavaFX 21.0.1 (controls, fxml)	Screens, tables, dialogs, animation
UI helpers	ControlsFX 11.1.2, FormsFX 11.6.0, ValidatorFX 0.4.0, BootstrapFX 0.4.0	Extra controls, forms, validation and Bootstrap-like styles
Database	MySQL 8 with Connector/J 8.0.33 (jar 8.2.0 also bundled)	Persistent storage over JDBC
Build	Maven (wrapper mvnw / mvnw.cmd), javafx-maven-plugin 0.0.8	mvn clean javafx:run
Modules	module-info.java: main.ahbab.main_interface	Java Platform Module System; packages opened to javafx.fxml
Testing	JUnit Jupiter 5.9.2 (configured)	Unit-test framework ready
IDE	IntelliJ IDEA (.idea files)	Development

6.2 Source packages

Package	Classes	Role
`main.ahbab.main_interface`	HomeScreen, HomeScreenController, UserController, UserSignUpController, ChatSystemController, SimpleServer	Entry point, login, sign-up, user chat, helper server
`.Admin`	AdminScreenController, HealthFeaturesController, EducationScreenController, OthersController, AdminCHATTER	Admin login and department consoles, admin chat
`.User`	GuestScreenController, HealthController, ParkingslotController, Tutor, Student	Dashboard and service screens
`.server`	ServerManager, ClientHandler	Socket server, authentication, message persistence, DB bootstrap
`.service`	UserService, HealthService	Reusable data-access services
`resources`	21 .fxml, 2 .css, images (logo, icons)	UI layouts and styling

6.3 Screen navigation map

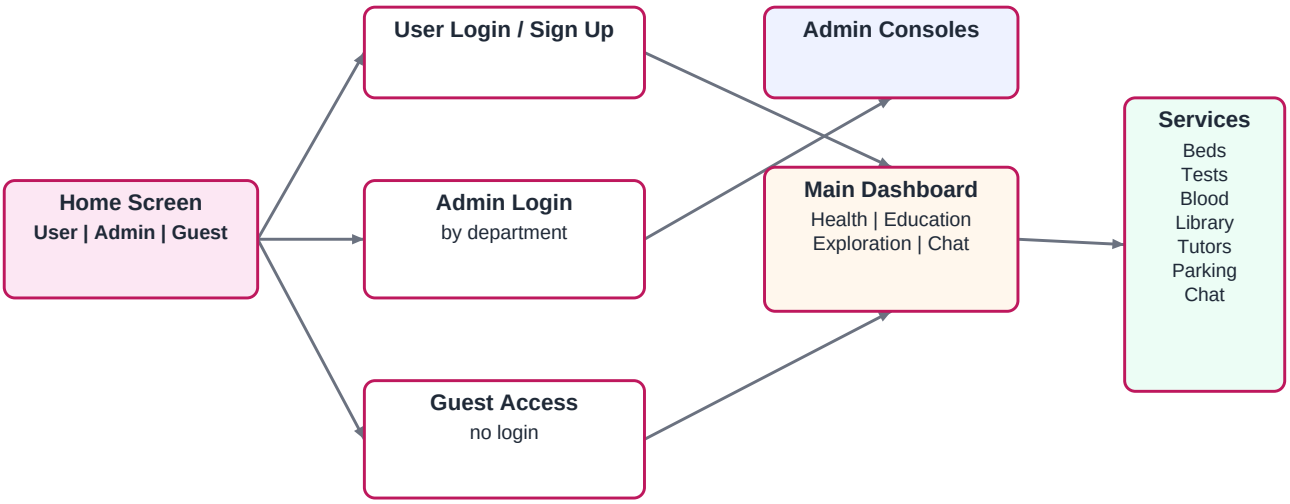


Figure 2 - Navigation between screens.

7. Chat and Socket Server

ServerManager is a thread-safe singleton. At start-up it connects to MySQL (creating `hex_database` and all tables if they do not exist), then listens on **TCP port 12345** and hands every connection to a **ClientHandler** running in a cached thread pool. Connected users are tracked in a `ConcurrentHashMap`. The protocol is line-based: `COMMAND:arg1:arg2` (split into at most three parts).

Command	Arguments	Reply	Status
`LOGIN`	username, password	SUCCESS:Login successful / ERROR:Invalid credentials	Implemented
`SEND_MESSAGE`	receiver, text	SUCCESS:Message sent (row inserted into messages)	Implemented

Command	Arguments	Reply	Status
`GET_MESSAGES`	chatWith	MESSAGES: sender: text (timestamp) lines	Implemented
`REGISTER`	username, password	SUCCESS:Registration successful	Placeholder
`GET_USERS`	-	USERS:user1,user2,user3	Placeholder
`HEALTH_DATA / EDUCATION_CONTENT`	-	SUCCESS:... processed	Placeholder
`PARKING_STATUS / RESERVE_PARKING`	slot	Fixed text replies	Placeholder
`DISCONNECT`	-	closes the session	Implemented

How chat behaves today

In the current user and admin chat screens the message text is saved straight into MySQL through JDBC (`SimpleServer.storeMessageInDatabase`) and the conversation view is reloaded after selecting a contact or sending. The socket connection is opened and ready, but new messages are not pushed automatically to the other party - the receiver sees them when the conversation is next loaded. Adding a push (server broadcasts to the registered ClientHandler) is the natural next step.

8. Database Design

Database **hex_database** (utf8mb4) is created by `schema_hex.sql` / `hex_database.sql`, or automatically by ServerManager on first run. Foreign keys use `ON DELETE CASCADE`.

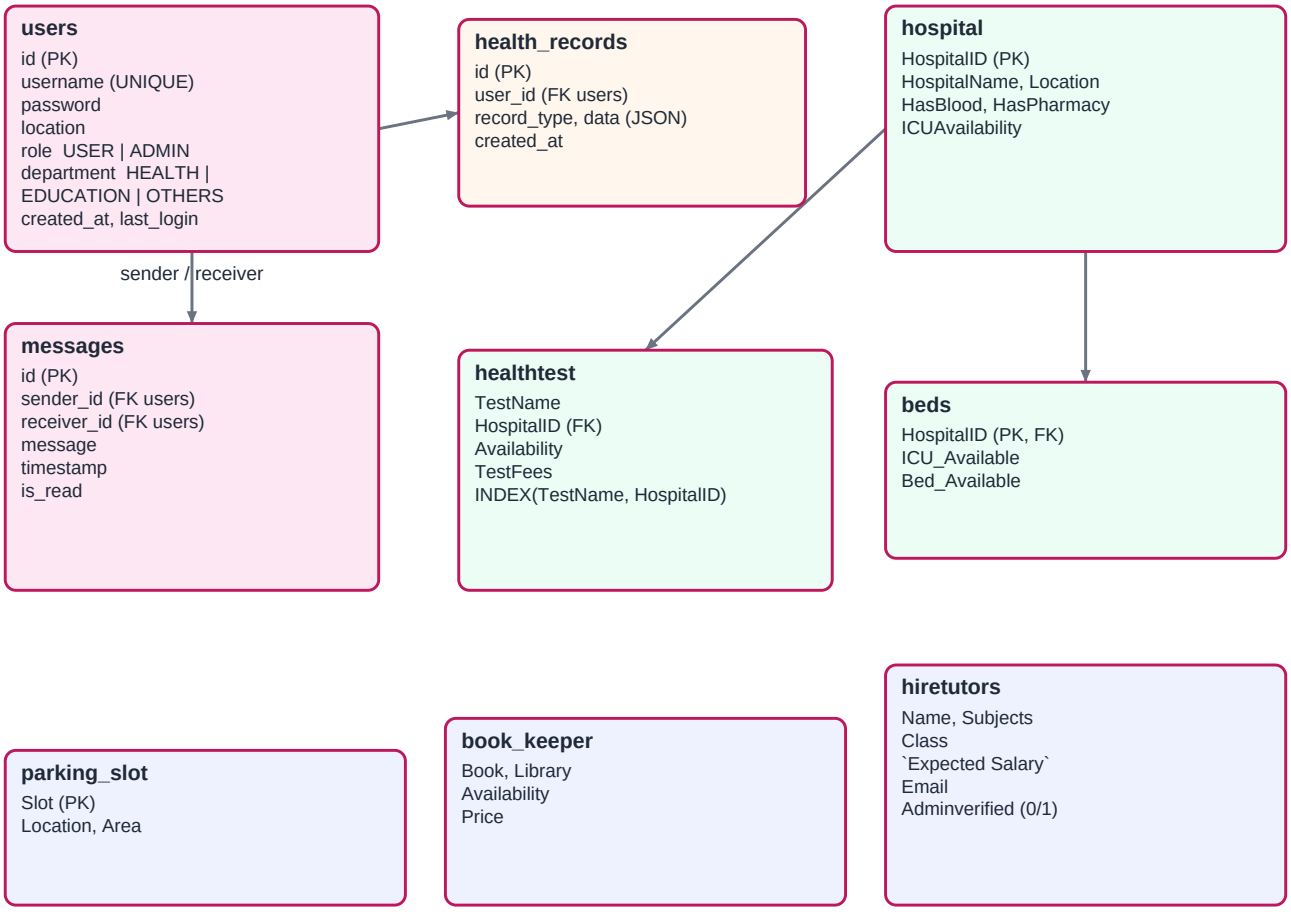


Figure 3 - Entity-relationship overview. Education and parking tables are standalone.

8.1 Data dictionary

Table	Purpose	Notes
`users`	Accounts for users and administrators	role + department drive administrator access; username is unique.
`messages`	Chat history	Two foreign keys to users; indexed by time order in queries.
`health_records`	Optional per-user JSON health data	Created by ServerManager and present in hex_database.sql; used by HealthService.
`hospital`	Hospital master data	Name, location, blood bank (Yes/No), pharmacy, ICU text status.
`beds`	Capacity per hospital	ICU_Available and Bed_Available counters; 1:1 with hospital.
`healthtest`	Test price list	One row per test per hospital; fee as DOUBLE.
`parking_slot`	Parking locations	Slot id, location and area.
`book_keeper`	Library catalogue	Book, library, availability, price.
`hiretutors`	Tutor directory	Adminverified flag controls visibility to students.

8.2 Seed data supplied

Table	Sample content
users	3 sample users and 5 administrators (2 Education, 2 Health, 1 Others)
hospital / beds	6 hospitals with bed and ICU counters
healthtest	25 prices: Ultrasound, MRI, COVID-19 PCR, Blood Culture Test, X-Ray for 5 hospitals
parking_slot	3 slots (Jatrabari, Malibag, Gulshan-2, Dhaka)
book_keeper	2 books
hiretutors	5 tutors, 3 to 4 verified (the two scripts differ slightly)

> **Data note**
> The hospital seed rows use US cities (New York, Los Angeles, Chicago, Houston, Miami, Orlando) while the parking rows are in Dhaka. For a Bangladesh deployment, replace the hospital, bed and test rows with local data. Demo account passwords in the SQL scripts are simple placeholders and must be changed.

9. Key Workflows

9.1 Find ICU beds

1. Open the app and choose **Guest Access**.
2. Click **Check Beds** under Health Services.
3. Type an area such as *Chicago* and press Search.
4. The table shows hospitals in that area with free beds and ICU units.

9.2 Administrator updates bed count

1. Choose **Admin Login**, enter a Health admin's credentials.
2. The system reads *department = HEALTH* and opens the Health console.
3. Enter hospital name and the new number, press update.
4. Query `UPDATE beds JOIN hospital ... SET Bed_Available=? WHERE HospitalName=?` runs and a success alert appears.
5. Users who search afterwards immediately see the new figure.

9.3 Verify a tutor

1. Education admin opens the Tutors table.
2. Selects/enters the tutor's name and presses **Verify** (`UPDATE hiretutors SET Adminverified = true WHERE Name = ?`).
3. The tutor now appears in the student's *Get Teachers* list.

9.4 Chat with an administrator

1. Sign in on the dashboard (the chat unlocks only after sign-in).
2. Open **Chat**; the administrator list loads.
3. Select an admin; history loads in the background.
4. Type and send; the message is stored in *messages* and the thread refreshes.

10. Installation and Run Guide

Prerequisites

- JDK 21 and (optionally) Maven - the wrapper `mvnw` is included.
- MySQL 8 running on `localhost:3306` with user `root` / password `root` (as configured in the code).
- Internet access on first run so Maven can download JavaFX and libraries.

Steps

1. Import the database: run `hex_database.sql` (full dump) or `src/main/resources/.../sql/schema_hex.sql` (compact schema + seed data) in MySQL. If neither is run, ServerManager creates an empty database and tables on first launch.
2. Open the folder `Main_Interface` in IntelliJ IDEA (or any terminal).
3. Run: `./mvnw clean javafx:run` (Windows: `mvnw.cmd clean javafx:run`). The plugin starts class `main.ahbab.main_interface.HomeScreen`.
4. Keep port 12345 free; the chat server starts automatically with the home screen.
5. Choose Guest Access to explore, or log in with a seeded account.

> **Configuration tip**
> If the MySQL password differs from **root**, update it in ServerManager and in each controller - the credentials are repeated in several classes (see Recommendation R2 in Section 11).

11. Security and Quality Review

11.1 Strengths

- Almost every data query uses **PreparedStatement** with bound parameters - good protection against SQL injection.
- `try-with-resources` closes connections and statements automatically.
- Long database work in chat runs on background threads; UI updates use `Platform.runLater`.
- Role and department separation of administrator functions.
- Clear separation of FXML (layout), CSS (style) and Java (logic); Java module descriptor with controlled exports.

11.2 Findings and recommendations

ID	Priority	Finding	Recommendation
R1	High	Passwords are stored and compared as **plain text** (users.password, authenticateUser).	Hash with BCrypt (jBCrypt) or PBKDF2; migrate existing rows.
R2	High	Database URL, user and password (root/root) are hard-coded in more than ten classes.	Create one DatabaseConfig (properties file / environment variables) and a connection pool such as HikariCP; use a restricted MySQL user.
R3	Medium	**Sign-up mismatch:** UserService.createUser inserts an *email* column, but users in schema_hex.sql / hex_database.sql has none.	Add `email VARCHAR(100)` to the SQL scripts (ServerManager already defines it).
R4	Medium	ServerManager shares one JDBC Connection between all client threads.	Open a connection per request or use a pool; Connection is not thread-safe.
R5	Medium	Developer Login screen references a handler (UserManager) that does not exist in code.	Implement it using UserService (getAllUsers, updateUser, deleteUser) and restrict to a developer role.
R6	Medium	Chat input over the socket is not authenticated for message storage; the sender name comes from a static variable (SimpleServer.loggedInUsername).	Route sending through the authenticated ClientHandler session and remove static state.
R7	Low	Sign-in stores the typed username in a static field before the password is checked.	Set it only after successful validation.
R8	Low	ServerManager and the legacy SimpleServer both target port 12345 (the second fails gracefully).	Keep one server implementation and delete the other.
R9	Low	No login attempt limiting; admin credentials follow a guessable pattern in seed data.	Throttle attempts; force password change on first login.
R10	Low	Sample hospital data are US locations while parking data are in Dhaka.	Load real local data.

12. Test Cases

ID	Test	Steps	Expected
T1	Guest access	Home > Guest Access	Dashboard opens without login
T2	Bed search	Check Beds, type "Florida"	Oceanview (Miami) and Cedars Sinai (Orlando) listed
T3	Test price	View Tests, type "MRI"	5 hospitals with fees 20 - 50
T4	Blood bank	Blood Services, type "Chicago"	Greenfield Clinic with HasBlood = Yes
T5	Tutor visibility	Find Tutors > Get Teachers	Only Adminverified tutors shown
T6	Admin routing	Log in as each department admin	Health, Education, Others consoles open respectively
T7	Bad login	Wrong password	"Invalid username or password" alert
T8	Bed update	Health admin updates a hospital's beds, then search as guest	New number visible
T9	Book CRUD	Education admin inserts, updates, deletes a book	Success alerts and updated library list
T10	Chat before login	Click Open Chat as guest	Message asking to sign in first
T11	Chat persist	User sends a message to an admin; admin opens chat	Message present in messages table and thread
T12	Sign-up validation	Passwords differ / under 6 chars	Warning alerts, no account created

13. Roadmap

Phase	Improvement	Benefit
1	Fix R1-R5; add email column; central configuration; BCrypt	Secure, runnable-everywhere baseline
2	Live push chat (server broadcasts to online clients); unread badges using is_read	True real-time communication
3	Tutor registration and profile, booking requests, ratings	Complete the education loop
4	Real parking reservation with time slots and status	Turn the prototype commands into features
5	Per-user health records and appointment reminders (HealthService is ready)	Personalisation
6	Map view of hospitals / parking, localisation (Bangla), data import from official sources	Usability and coverage
7	Web / mobile front end using the same database through a REST API	Wider reach

14. What We Built - Team and Deliverables

The Java package name and the in-app Contact dialog identify **Md Ahbab Hamid Khan** as the lead developer. Add your team members and their responsibilities in the table.

Team member	Responsibility
Md Ahbab Hamid Khan	Lead developer - application, database and server code (add specific modules)
[Teammate name]	[role, e.g. UI design / database / testing]
[Teammate name]	[role]

Deliverable	Detail
Source code	20 Java classes (about 3,400 lines), module descriptor, Maven build
User interface	21 FXML screens, 2 stylesheets, logo and icons
Database	Compact schema + seed script and a full dump (9 tables)
Server	Multi-threaded socket server with a command protocol
Help	Contact, bug-report and About dialogs; online About document

15. Presentation Guide and Viva Questions

15.1 Suggested 7-minute demo

1. **Idea (45 s)** - one app for health, education and exploration.
2. **Guest tour (2 min)** - beds search, test price comparison, blood bank, library, parking.
3. **Verified tutors (45 s)** - show that only verified tutors are visible.
4. **Admin (2 min)** - log in as Health admin, change a bed count; log in as Education admin, verify a tutor.
5. **Chat (45 s)** - user sends a message, admin sees it.
6. **Architecture and roadmap (45 s)** - layers, socket server, next steps.

15.2 Likely questions and model answers

Q. Why JavaFX instead of a web app?

JavaFX gives a rich native interface, FXML separates layout from code, and the app can run on any desktop with a JVM. The same database can later serve a web or mobile front end.

Q. What is FXML and how is it linked to Java?

FXML is an XML description of the screen. Each file names a controller class with fx:controller; fields with @FXML are injected and onAction handlers call controller methods.

Q. How do you avoid freezing the UI during database work?

Long queries (chat history) run in a new Thread and update controls through Platform.runLater, because JavaFX controls may be changed only on the Application Thread.

Q. How are administrators restricted to their department?

The users table has role and department. After login the department string decides which FXML console loads, and each console only contains actions on its own tables.

Q. What is the role of the socket server?

It accepts TCP clients on port 12345, authenticates with LOGIN and processes commands such as SEND_MESSAGE using a thread pool. Today chat messages are persisted directly; the server is the base for real-time push.

Q. Why is ServerManager a singleton?

There must be exactly one server, one thread pool and one connected-clients registry for the whole app.

Q. How did you prevent SQL injection?

All queries with user input use PreparedStatement parameters. We list the remaining security work (password hashing, configuration) in Section 11.

Q. How does the dynamic TableView work?

Columns are created from ResultSetMetaData names and each row is a Map of column name to value, so the same code can display any query.

Q. What is JPMS (module-info.java) used for?

It declares dependencies (requires javafx.*, java.sql ...) and which packages are opened to FXML reflection.

Q. What are the limitations?

Plain-text passwords, repeated credentials, placeholder socket commands, US sample data, and no tutor registration. All are captured with fixes in the review and roadmap.

Q. How would you scale it?

Move logic to a REST service with connection pooling, hash passwords, use WebSocket push for chat, and serve a web and mobile client.

Appendix - Glossary

Term	Meaning
JavaFX / FXML	Java UI toolkit and its XML screen-layout format
JDBC	Java Database Connectivity - API for SQL databases
Prepared statement	SQL with placeholders so user input can never change the query structure
Singleton	A class that allows only one instance
Thread pool	Reusable worker threads for concurrent client connections
TableView	JavaFX control displaying rows and columns
JPMS	Java Platform Module System (module-info.java)
ICU	Intensive Care Unit